CI/CD HANDS-ON LAB | JOB 2 | BEGINNER

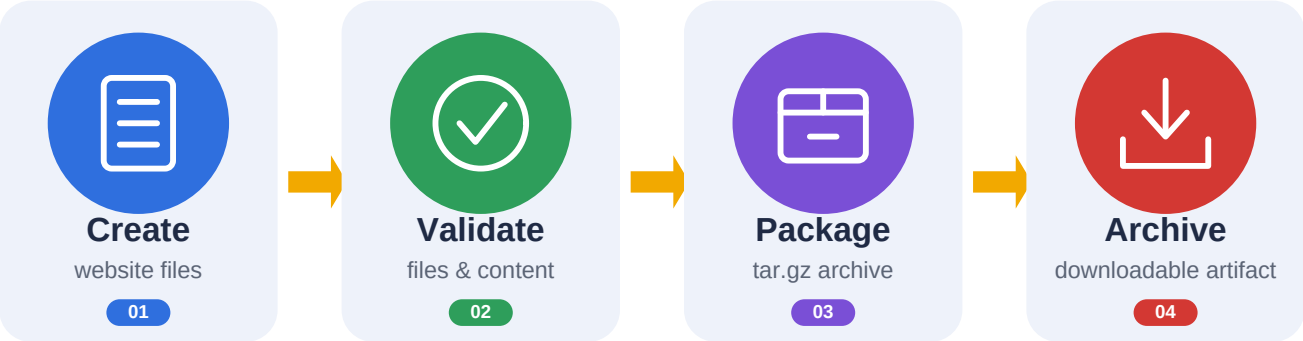
Build a Mini Project from Scratch

Create, validate and package a website, all from the Jenkins UI

Freestyle job | 3 build steps | 1 artifact

This time Jenkins does more than print messages. You will simulate a real CI build: generate a small website, check that it is valid, package it, and keep the result as a downloadable artifact. No Jenkinsfile required.

1 What will we build?



Job name: `CloudMessenger-Website-Build`

What you'll learn

Workspace | Build steps | Shell commands | Build status | Console logs | Artifacts

2 Set up the job

1

Create the job

- Open the Jenkins dashboard and click **New Item**.
- Name it `CloudMessenger-Website-Build`
- Select **Freestyle project** and click **OK**.

2

General settings

- Description: *Build, validate, and package a sample website using Jenkins Freestyle Project.*
- Leave other options at their defaults.

3

Source Code Management

- Select **None**; leave Build Triggers unchecked.
- The website is created directly in the Jenkins workspace.
- Later, you can connect a Git repository.

3 Build steps: your 3 shell scripts

Scroll to **Build Steps**, click **Add build step**, and choose **Execute shell** once for each script below.

STEP 4 Build Step 1: Create the website

```
#!/bin/bash
set -eu

echo "STEP 1: Creating website files"

mkdir -p website

cat > website/index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Cloud Messenger</title>
</head>
<body>
    <h1>Welcome to Cloud Messenger!</h1>
    <p>My second Jenkins Freestyle project.</p>
    <p>Learn. Build. Deploy. Grow.</p>
</body>
</html>
EOF

cat > website/README.txt <<'EOF'
Cloud Messenger Demo Website
Created using Jenkins Freestyle Project.
EOF

echo "Website files created successfully."
```

Understand the commands

Command	Meaning
`set -eu`	Stop the script when a command fails or an undefined variable is used
`mkdir -p website`	Create the website directory
`cat > website/index.html`	Write HTML into the file
`cat > website/README.txt`	Create a README file
`EOF`	Marks the end of the text being written
`echo`	Prints a message in the build log

STEP 5 Build Step 2: Validate the website

Click **Add build step** then **Execute shell** again, and enter:

```
#!/bin/bash
set -eu

echo "STEP 2: Validating website files"

test -s website/index.html
test -s website/README.txt

grep -q "Welcome to Cloud Messenger!" website/index.html

echo "Website validation passed."
echo "Files available:"
ls -lh website/
```

`test -s`	checks that a file exists and is not empty
`grep -q`	checks the expected text exists in the HTML
`ls -lh`	lists the generated files

If validation fails
Jenkins should mark the build as **failed**.

STEP 6 Build Step 3: Package the website

Click **Add build step** then **Execute shell** one more time, and enter:

```
#!/bin/bash
set -eu

echo "STEP 3: Packaging website"

tar -czf website-build.tar.gz website/

echo "Package created successfully."
ls -lh website-build.tar.gz
```

This creates a compressed archive of your website files. Anatomy of the command:

`tar`	`-c`	`-z`	`-f`
packages files	create an archive	compress with gzip	set the filename

The output file is `website-build.tar.gz`.

4 Archive the artifact

STEP 7 Post-build Action

- Scroll down to **Post-build Actions** and click **Add post-build action**.
- Select **Archive the artifacts**.
- In **Files to archive**, enter `website-build.tar.gz`
- Leave the other options at their defaults.

> **Why archive artifacts?**
> Jenkins saves the generated package with the build record, so you can download it later instead of searching through the workspace.

5 Run it and inspect the result

STEP 8 Save and execute

- Click **Save**, then **Build Now** on the job page.
- Wait for the build to finish.
- Under Build History, click the latest build number such as **#1**.
- Open **Console Output**. Your logs should include:

```
STEP 1: Creating website files
Website files created successfully.
STEP 2: Validating website files
Website validation passed.
STEP 3: Packaging website
Package created successfully.
Finished: SUCCESS
```

The exact output will include additional Jenkins log messages.

STEP 9 Download the artifact

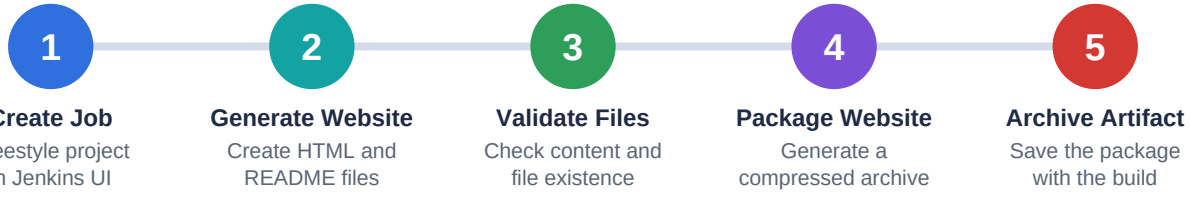
- Open the job's build page.
- Find **Build Artifacts**.
- Click `website-build.tar.gz` to download it.

The archive should contain:

```
website/
├── index.html
└── README.txt
```

You have now created and validated a website and packaged it using Jenkins.

6 The complete workflow



7 Final checklist

- [ ] Created CloudMessenger-Website-Build
- [ ] Added the website creation build step
- [ ] Added the validation build step
- [ ] Added the packaging build step
- [ ] Configured artifact archiving
- [ ] Ran the job and confirmed SUCCESS
- [ ] Downloaded the archived artifact

Important
This is a beginner CI-style build exercise, not yet a complete production CI/CD pipeline.

Next step: create a third Freestyle job that checks out code from GitHub, runs a build, and archives the output. That introduces source control integration.